

Branch Prediction



John Jose

Associate Professor

Department of Computer Science & Engineering

Indian Institute of Technology Guwahati

Limits to pipelining

- ❖ **Hazards:** circumstances that would cause incorrect execution if next instruction is fetched and executed
 - ❖ **Structural hazards:** Different instructions, at different stages, in the pipeline want to use the same hardware resource
 - ❖ **Data hazards:** An instruction in the pipeline requires data to be computed by a previous instruction still in the pipeline
 - ❖ **Control hazards:** Succeeding instruction, to put into pipeline, depends on the outcome of a previous branch instruction, already in pipeline

Four Branch Hazard Alternatives

#1: Stall until branch direction is clear

#2: Predict Branch Not Taken

#3: Predict Branch Taken

#4: Delayed Branch

Four Branch Hazard Alternatives

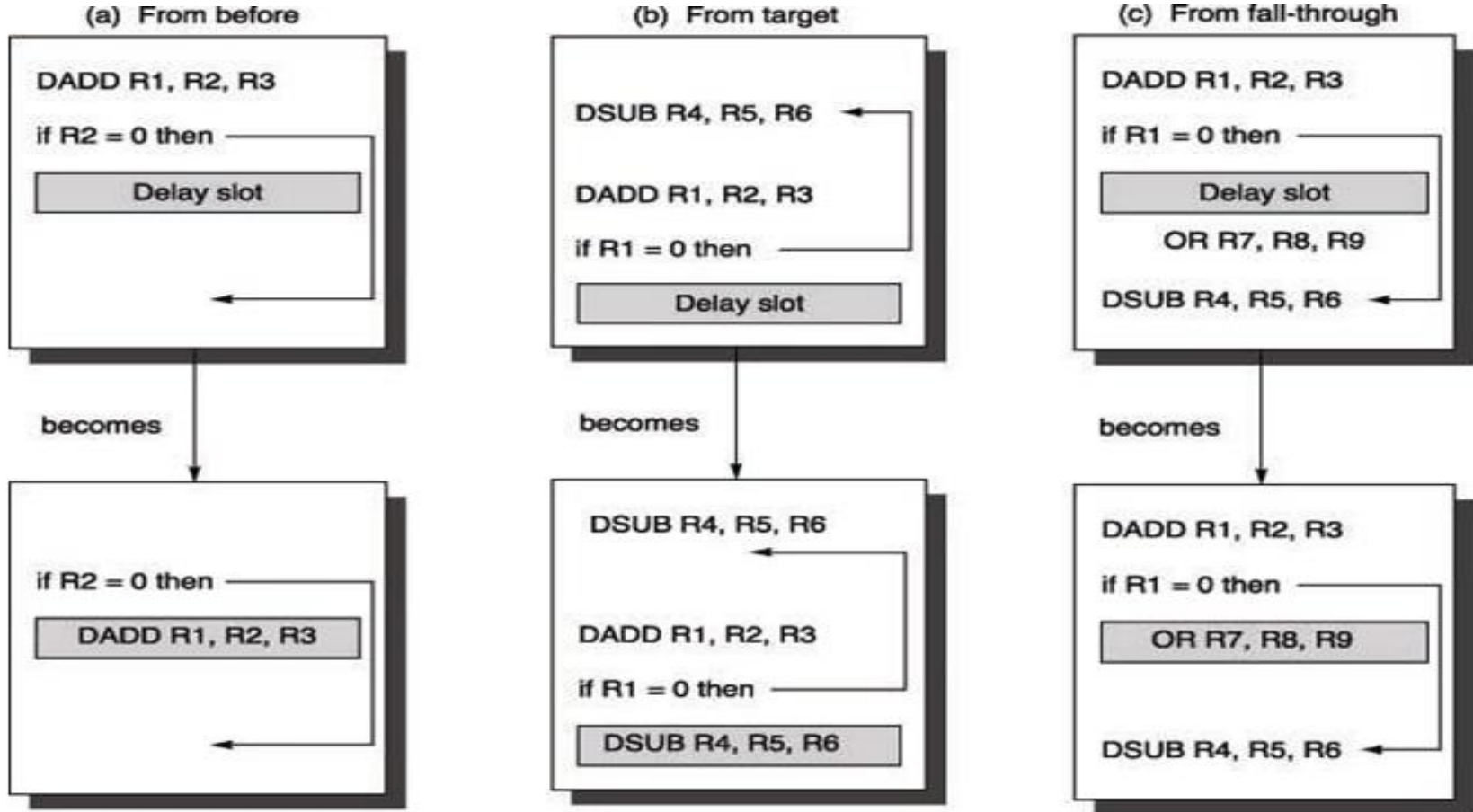
#4: Delayed Branch

- ❖ Define branch to take place **AFTER** one instruction following the branch instruction.
- ❖ 1 slot delay allows proper decision and branch target address in 5 stage pipeline (MIPS uses this approach)
- ❖ Where to get instructions to fill branch delay slot?

Untaken branch instruction	IF	ID	EX	MEM	WB				
Branch delay instruction ($i + 1$)		IF	ID	EX	MEM	WB			
Instruction $i + 2$			IF	ID	EX	MEM	WB		
Instruction $i + 3$				IF	ID	EX	MEM	WB	
Instruction $i + 4$					IF	ID	EX	MEM	WB

Taken branch instruction	IF	ID	EX	MEM	WB				
Branch delay instruction ($i + 1$)		IF	ID	EX	MEM	WB			
Branch target			IF	ID	EX	MEM	WB		
Branch target + 1				IF	ID	EX	MEM	WB	
Branch target + 2					IF	ID	EX	MEM	WB

Filling Branch Delay Slot



Conditional Branches

- ❖ When do you know you have a branch?
 - ❖ During ID cycle (Could you know before that?)
- ❖ When do you know if the branch is Taken or Not-Taken
 - ❖ During EXE cycle/ ID stage depending on the design
- ❖ We need for sophisticated solutions for following cases
 - ❖ Modern pipelines are deep (10 + stages)
 - ❖ Several instructions issued/cycle
 - ❖ Several predicted branches in-flight at the same time

Dynamic branch prediction

- ❖ Execution of a branch requires knowledge of:
 - ❖ **Branch instruction** - encode whether instruction is a branch or not. Decide on taken or not taken (i.e., prediction can be done at IF stage)
 - ❖ Whether the **branch is Taken/Not-Taken** (hence a branch prediction mechanism)
 - ❖ If the branch is taken what is the **target address** (can be computed but can also be “precomputed”, i.e., stored in some table)
 - ❖ If the branch is taken **what is the instruction at the branch target address** (saves the fetch cycle for that instruction)

Dynamic branch prediction

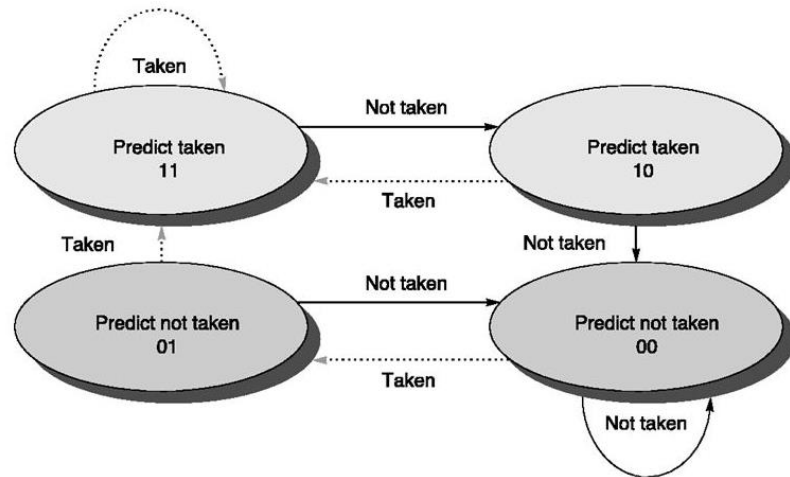
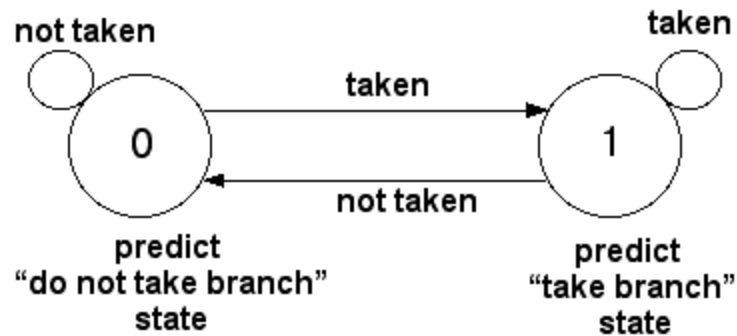
- ❖ Use a Branch Prediction Buffer (BPB)
 - ❖ Also called Branch Prediction Table (BPT), Branch History Table (BHT)
 - ❖ Records previous outcomes of the branch instruction.
 - ❖ Used to predict the likely outcome of future branches based on this history
- ❖ A prediction using BPB is attempted when the branch instruction is fetched (IF stage or equivalent)
- ❖ It is acted upon during ID stage (when we know we have a branch)

Dynamic branch prediction

- ❖ Has a prediction **been made** (Y/N)
 - ❖ If not use default “Not Taken”
- ❖ Is it **correct or incorrect** ?
- ❖ Two cases:
 - ❖ Case 1: Yes and the prediction was correct (known at ID stage) or No but the default was correct: No delay
 - ❖ Case 2: Yes and the prediction was incorrect or No and the default was incorrect: Delay

Prediction Scheme with 1 or 2 bit FSM

- ❖ The use of a 2-bit predictor will allow branches that favor taken (or not taken) to be mispredicted less often than the one-bit case. (reinforcement learning)



Branch Prediction In Hardware

- ❖ Branch prediction is extremely useful in loops.
- ❖ A simple branch prediction can be implemented using a small amount of memory indexed by lower order bits of the address of the branch instruction. **(branch prediction buffer)**
- ❖ One bit stores whether the branch was taken or not.
- ❖ The next time the branch instruction is fetched refer this bit

Advanced Branch Prediction Techniques

❖ Correlating predictor: (m, n)

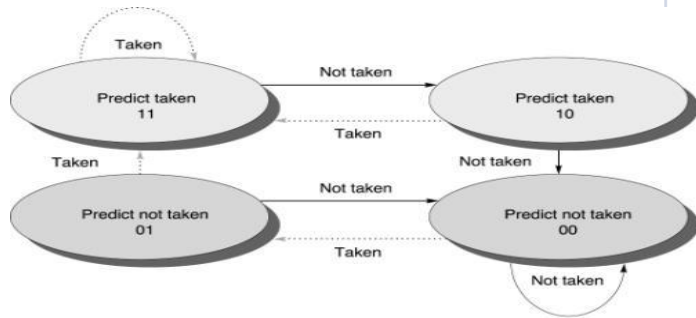
- ❖ Multiple n-bit predictors for each branch
- ❖ One for each possible combination of outcomes of preceding m branches

```
if ( x == 2) /* br-1*/  
    x = 0;  
if ( y == 2) /* br-2*/  
    y = 0;  
if ( x != y) /* br-3/  
do this  
else do that
```

Branch Prediction - Example

Consider the last 5 actual outcomes of a single static branch. T means branch is taken and N means not taken. {oldest $\rightarrow$ T T N N T $\leftarrow$ latest}

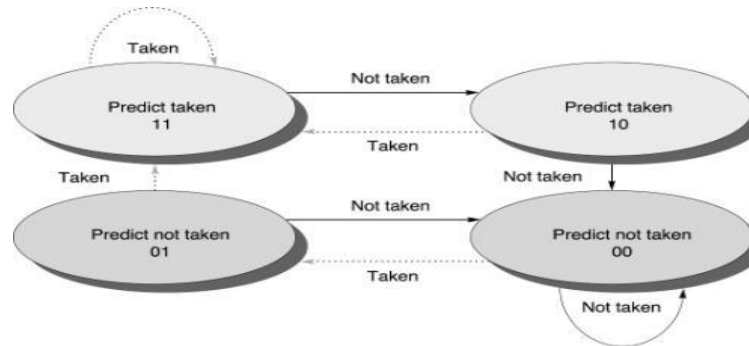
A two level (1,2) type correlating branch predictor is used. Last branch outcome is used to index to the BHT which is initialized to 00 and 11 for N and T, respectively. How many mis-predictions are there and which of the branches in this sequence would be mis-predicted? Fill up the table for 5 branch outcomes.



Sl.No	Last Outcome	BHT N/T	Prediction	Outcome	Mis-Pre Y/N ?
1	N (initial)	00 / 11	N	T	YES
2	T	01 / 11	T	T	NO
3	T	01 / 11	T	N	YES
4	N	01 / 10	N	N	NO
5	N	00 / 10	N	T	YES
		01 / 10			

Branch Prediction – Example 2

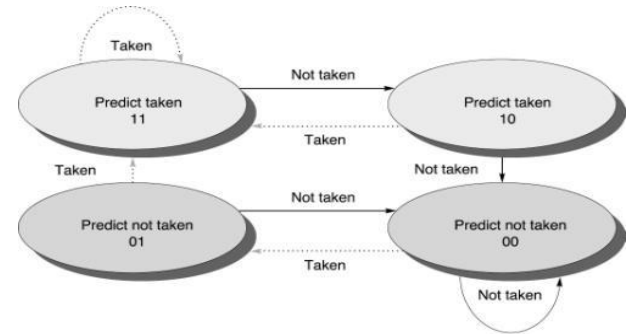
Consider a (2,2) type branch predictor. BHT is indexed by the outcome of the last 2 branches. The BPB is initialized for NN/NT/TN/TT as 00/00/11/11 and is indexed with an NN entry in the first reference. Consider the last 8 actual outcomes of a single static branch, {oldest N N T T T N N T latest} where T means branch is taken and N means not taken. What will be the contents of BPB after the execution of the above mentioned 6 branch outcomes?



Branch Prediction – Illustration

(2,2) type branch predictor. Initial: NN/NT/TN/TT - 00/00/11/11
Last 8 actual outcomes, {oldest N N T T T N N T latest}

S. No.	Last Outcome	BPB NN/NT/TN/TT	Prediction	Outcome	Misprediction Y/N?
1.	NN (Initial)	00/00/11/11	N	N	No
2.	NN	00/00/11/11	N	N	No
3.	NN	00/00/11/11	N	T	Yes
4.	NT	01/00/11/11	N	T	Yes
5.	TT	01/01/11/11	T	T	No
6.	TT	01/01/11/11	T	N	Yes
7.	TN	01/01/11/10	T	N	Yes
8.	NN	01/01/10/10	N	T	Yes
		11/01/10/10			





johnjose@iitg.ac.in
<http://www.iitg.ac.in/johnjose/>